

## Project 3

Ryley Nathaniel

Department of Psychology, York University

PSY6136: Categorical Data Analysis

Dr. Michael Friendly

April 2026

### Table of Contents

|                                 |   |
|---------------------------------|---|
| 1. Abstract.....                | 1 |
| 2. Introduction .....           | 2 |
| 3. Methods.....                 | 2 |
| 3.1 Streaming Framework .....   | 2 |
| 3.2 Asymptotic Properties ..... | 2 |
| 3.4 Incremental Algorithm.....  | 3 |
| 4. Simulation Study .....       | 3 |
| 5. R Package Structure .....    | 3 |
| 5.1 Proposed Package.....       | 3 |
| 5.2 Core Functions.....         | 3 |
| 5.3 Object Structure .....      | 4 |
| 6. Discussion.....              | 4 |
| 7. Conclusion .....             | 4 |
| 8. References .....             | 5 |
| 9. Appendix .....               | 6 |

## 1. Abstract

Classical categorical data analysis (CDA) methods assume static datasets in which contingency tables are constructed after full data collection. However, modern applications, such as clickstream monitoring, online surveys, and real-time behavioural tracking, generate categorical observations sequentially. I propose an incremental CDA framework that updates contingency tables, association measures, and visualizations in real time as new observations arrive. My approach maintains exact cell counts and marginal totals through streaming updates, allowing dynamic computation of chi-square statistics, standardized residuals, and Cramer's V without recomputing the full dataset. I further introduce dynamic mosaic visualizations that reveal evolving association structures. Simulation studies demonstrate computational efficiency and stable convergence of association estimates. This framework enables real-time monitoring of categorical relationships in high-velocity data environments.

*Keywords:* CDA, static dataset, contingency table, dynamic computation

# REAL-TIME CATEGORICAL DATA ANALYSIS

## 2. Introduction

Classical categorical data analysis (CDA) methods – such as chi-square tests of independence, log-linear models, and correspondence analysis – are formulated under a static sampling paradigm (Agresti, 2002; Agresti, 2018). This involves a contingency table that is constructed after all observations are collected, and an inference that proceeds conditionally on a fixed total sample size (Agresti, 2002; Agresti, 2018). However, many modern data-generating processes are inherently sequential. This makes these conditions difficult to satisfy. Examples such as online transactions, clickstream events, adaptive experiments, streaming surveys, and digital behavioural monitoring produce categorical observations continuously over time. In such environments, waiting until data collection terminates before assessing association is often not an overly practical approach. Instead, analyses demand real-time monitoring of categorical dependence.

Despite extensive classical and recent research in streaming regression, online principal component analysis (PCA), and sequential change detection, comparatively little attention has been given to incremental inference from contingency tables (Bifet & Gavaldà, 2009; Friedman, 2001; Marrón et al., 2019). The current paper develops a theoretical and practical framework for online estimation and monitoring of categorical association under sequential sampling. I formalize contingency table counts as stochastic processes indexed by time and analyze the asymptotic behaviour of streaming chi-square statistics and related association measures.

My contributions are threefold where I:

1. Formalize contingency table inference under sequential observation.
2. Establish asymptotic behaviour of cumulative chi-square statistics.
3. Provide computationally efficient algorithms for real-time updating and visualization.

This framework unifies classical categorical inference with modern streaming analytics and provides statistically grounded tools for monitoring evolving dependence structures.

## 3. Methods

This section outlines the methodological framework for extending CDA to a real-time setting.

Instead of analyzing a fixed dataset, we allow contingency tables and associated statistics to update continuously as new observations arrive.

### 3.1 Streaming Framework

As aforementioned, classical CDA is based on a fixed dataset, where a contingency table is constructed once and analyzed (Agresti, 2002; Agresti, 2018). In contrast, a streaming framework treats data as arriving sequentially over time (Aggarwal, 2007; Domingo & Hulten, 2000; Gama, 2010).

In a streaming framework, each new observation consists of a pair of categorical values (e.g.,  $X$  and  $Y$ ) that contributes to a specific cell in a contingency table. Rather than rebuilding the table from scratch, we update the existing table by incrementing the corresponding cell count. This produces a sequence of contingency tables that evolve over time.

This approach allows us to monitor how relationships between categorical variables change as more data are collected. It is especially useful in settings where data are naturally sequential, such as the work I do with behavioural experiments and/or neural recordings across trials. Instead of single static summary, we obtain a dynamic view of association.

### 3.2 Asymptotic Properties

As the number of observations increases, the estimates derived from the contingency table become more stable and reliable (Agresti, 2013; Wasserman, 2004). Early in the data stream, results may fluctuate due to small sample sizes, but these fluctuations decrease as more data accumulate (Wasserman, 2004).

As it pertains to the relationship between variables, if the two categorical variables are truly independent, the observed cell counts will gradually resemble the expected counts under independence. On the other hand, if there is a genuine association, the deviation between observed and expected counts will become more consistent over time.

This behaviour is important in a streaming context because it helps distinguish noise from meaningful structure. While early results should be interpreted cautiously, later results provide stronger evidence about the underlying relationship between variables. In this way, the streaming approach retains the same long-run reliability as classical CDA while

adding the ability to observe how conclusions develop over time.

### 3.4 Incremental Algorithm

To support real-time analysis efficiently, I implemented an incremental algorithm that updates results as new data arrive without recomputing everything from scratch. Instead, each incoming observation modifies only a single cell in the contingency table, allowing for fast updates to counts and derived statistics.

When using a full cumulative table, this involves simply increasing relevant cell count and updating summary measures. The incremental approach significantly reduces computational cost, making it feasible to apply CDA in real time, even for long data streams. It also enables continuous monitoring of key statistics, allowing researchers to visualize trends and detect change-points.

## 4. Simulation Study

I designed a simulation study using *R* code (see Appendix or the separate *Rmd.* file I sent via email).

The batch information is seen in Table 1, while the final contingency table is seen in Table 2. Mosaic plots and heatmaps (see Figure 1 simulation results) were generated for each batch number.

**Table 1**

*Batch statistics*

| Batch # | Chi-sq | Cramer's V |
|---------|--------|------------|
| 1       | NaN    | NaN        |
| 2       | 7.17   | 0.60       |
| 3       | 7.88   | 0.51       |
| 4       | 5.83   | 0.38       |
| 5       | 6.13   | 0.35       |
| 6       | 5.84   | 0.31       |
| 7       | 3.53   | 0.22       |
| 8       | 3.51   | 0.21       |
| 9       | 3.81   | 0.21       |
| 10      | 8.21   | 0.29       |
| 11      | 7.93   | 0.27       |
| 12      | 7.76   | 0.25       |
| 13      | 7.81   | 0.25       |
| 14      | 7.44   | 0.23       |
| 15      | 4.92   | 0.18       |
| 16      | 3.84   | 0.15       |
| 17      | 3.30   | 0.14       |
| 18      | 2.80   | 0.12       |
| 19      | 3.64   | 0.14       |
| 20      | 4.06   | 0.14       |

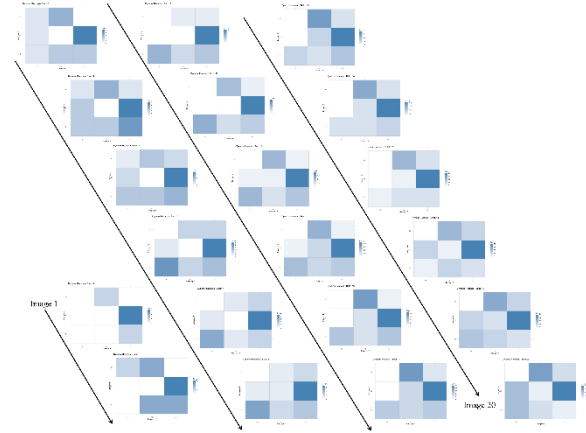
**Table 2**

*Final contingency table*

|    | B1 | B2 | B3 |
|----|----|----|----|
| A1 | 12 | 12 | 8  |
| A2 | 10 | 9  | 14 |
| A3 | 9  | 16 | 10 |

**Figure 1**

*Heatmap progression*



*Note.* Heatmap progression from image 1 to image 20. Image 1 is generated in the first batch while image 20 is generated in the final one. All the frames in between are for image 2-19. Each image number corresponds to a respective batch number as seen in Table 1. Images are numbered based on their position, read left to right.

## 5. R Package Structure

### 5.1 Proposed Package

Proposed package: *streamCDA*

This package is designed to maintain and monitor categorical association in real time. It implements incremental updating of contingency tables, cumulative and association statistics, real-time diagnostics, and visualization tools. The internal object stores cell count matrix, row marginals, column marginals, and total observations. All core functions can be seen in Table 2.

### 5.2 Core Functions

**Table 3**

### Core functions

| Name                                                         | Purpose                                                                                                                            | Inputs                                                                                                            | Outputs                                                                                                                                                                                                   |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>init_stream</b><br>( <i>k</i> , <i>m</i> )                | Initializes a streaming contingency table object. Creates the internal structure that will hold all running counts and statistics. | <i>k</i> – number of categories for variable <i>X</i><br><i>m</i> – number of categories for variable <i>Y</i>    | <i>counts</i> – <i>k</i> × <i>m</i> matrix initialized to zero<br><i>row_totals</i> – length <i>k</i> vector<br><i>col_totals</i> – length <i>m</i> vector<br><i>n</i> – total observations (initially 0) |
| <b>update_stream</b><br>( <i>obj</i> , <i>x</i> , <i>y</i> ) | Updates the contingency table with one new observation. Core streaming operation.                                                  | <i>obj</i> – streaming object<br><i>x</i> – category index for <i>X</i><br><i>y</i> – category index for <i>Y</i> | The updated streaming object ( <i>obj</i> ) with modified internal counts and metadata.                                                                                                                   |
| <b>chisq_stream</b><br>( <i>obj</i> )                        | Computes the cumulative chi-square statistic at time <i>t</i> .                                                                    | <i>obj</i> – streaming object                                                                                     | <i>statistic</i> – chi-square value<br><i>df</i> – degrees of freedom<br><i>p_value</i> – asymptotic p-value                                                                                              |
| <b>cramers_v_stream</b><br>( <i>obj</i> )                    | Computes Cramer's V at time <i>t</i> .                                                                                             | <i>obj</i> – streaming object                                                                                     | Single numeric value between 0 and 1.                                                                                                                                                                     |
| <b>plot_mosaic_stream</b><br>( <i>obj</i> )                  | Produces dynamic mosaic plot.                                                                                                      | <i>obj</i> – streaming object                                                                                     | Graphical mosaic plot representing the current contingency table stored in <i>obj</i> .                                                                                                                   |
| <b>plot_residual_heatmap</b><br>( <i>obj</i> )               | Displays heatmap of standardized residuals.                                                                                        | <i>obj</i> – streaming object                                                                                     | - Colored grid<br>- Red/blue shading for positive/negative residuals<br>- Optional threshold highlighting                                                                                                 |

### 5.3 Object Structure

For object structure, I am suggesting S3 or R6 class storing. Both S3 and R6 are ways of organizing data and functions together in R. They differ in how they behave. In the case of real-time CDA, S3 and R6 would store the state of the streaming analysis. They are containers (objects) that hold everything needed to keep the analysis running as new data arrives. For the streaming CDA framework, the object stores: (1) count matrix, (2) marginals, (3) total, and (4) history.

For the count matrix we are looking at a contingency table (core data). This is a matrix of counts for each combination of categories. This is the main object being updated in real time. Marginal totals are row sums and column sums. These are useful for computing expected counts quickly. Total sample size (*n*) is the number of observations processed so far. This updates with every new data point. History stores past values of statistics (e.g., chi-square over time). This enables plotting trends.

## 6. Discussion

This work extends classical contingency table analysis into the sequential domain. By treating cell counts as stochastic processes indexed by time, I established convergence properties of streaming association measures and provided monitoring tools.

The framework naturally connects categorical data analysis with sequential testing and change-point detection. It also provides a foundation for future extensions, including online log-linear modeling, streaming correspondence analysis, and high-dimensional categorical monitoring.

It must be emphasized that this does not simply mean that we update a table, it means that we introduce online monitoring of categorical dependency, real-time association diagnostics, streaming inference, and temporal association dynamics. Thus, the suggested framework is at the intersection of categorical data analysis, streaming analytics, online inference, and visual analytics.

Limitations include sensitivity to sparse categories and computational scaling in very high-dimensional tables. Future research may explore regularization strategies or sketching-based approximations (Aggarwal, 2007; Tibshirani, 1996).

## 7. Conclusion

I developed a theoretical and computational framework for real-time monitoring of categorical association under sequential sampling. I demonstrated that cumulative chi-square statistics exhibit distinct asymptotic behaviour under independence and dependence, providing a principled basis for streaming inference. This work bridges classical categorical data analysis with modern streaming data environments and opens avenues for dynamic categorical modeling in high-velocity applications.

## 8. References

- Aggarwal, C. C. (2007). *Data Streams: Models and Algorithms*. Springer.
- Agresti, A. (2002). *Categorical Data Analysis* (2<sup>nd</sup> ed.). Wiley.
- Agresti, A. (2013). *Categorical Data Analysis* (3<sup>rd</sup> ed.). Wiley.
- Agresti, A. (2018). *An introduction to categorical data analysis* (3<sup>rd</sup> ed.). Wiley.
- Bifet, A., & Gavalda, R. (2009). Adaptive learning from evolving data streams. *Advances in Intelligent Data Analysis VIII*, 5772, 249-260. [https://doi.org/10.1007/978-3-642-03915-7\\_22](https://doi.org/10.1007/978-3-642-03915-7_22)
- Domingos, P., & Hulten, G. (2000). Mining high-speed data streams. *KDD*, 71-80. <https://doi.org/10.1145/347090.347107>
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189-1232. <https://www.jstor.org/stable/2699986>
- Gama, J. (2010). *Knowledge Discovery from Data Streams*. Chapman & Hall/CRC.
- Marrón, D., Ayguadé, E., Herrero, J. R., & Bifet, A. (2019). Resource-aware elastic swap random forest for evolving data streams. *arXiv*. <https://arxiv.org/abs/1905.05881>
- Tibshirani, R. (1996). Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1), 267-288. <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>
- Wasserman, L. (2004). *All of Statistics*. Springer.

## AI Disclosures

I used ChatGPT (<https://chatgpt.com/>) to help with the following situations:

1. I used AI to help me generate a function to help with updating contingency tables. Although I have extensive experience coding functions in MATLABs, I had never done so in R, thus, I needed help with the formatting. To do this, I used ChatGPT and searched, “function to update contingency table in R” and worked from that prompt.
2. I used AI again when incorporating the function into dynamic plotting. From previous course experience, I knew how to plot the mosaics and heatmaps, but making them functional was a different story. To learn how to do this, I put my normal mosaic and heatmap plot codes into ChatGPT and asked, “how to incorporate the afore-used function for dynamic plotting with this code”.

## 9. Appendix

Full R Script (also attached as html file)

```
#' ---
#' title: "Project 3"
#' author: "Ryley Nathaniel"
#' date: "April 2026"
#' ---

#
=====

# Real-Time / Incremental
# Categorical Data Analysis
#
=====

# Load required packages
library(vcd)          # For mosaic
plots
library(ggplot2)      # For heatmaps
library(reshape2)     # For reshaping
tables

# -----

# 0. Output folder for figures
# -----

output_folder <-
"C:/Users/ryleyn/Documents/1uni/1ma/
categorical/my6136-master/images"

if(!dir.exists(output_folder)) {
  dir.create(output_folder,
recursive = TRUE)
}

# -----

# 1. Initialize categories and
# contingency table
```

```
# -----

categories_A <- c("A1", "A2", "A3")
categories_B <- c("B1", "B2", "B3")

cont_table <- matrix(0,
                      nrow =
length(categories_A),
                      ncol =
length(categories_B),
                      dimnames =
list(categories_A, categories_B))

# -----

# 2. Function to update contingency
# table
# -----

update_contingency <-
function(new_data, table) {
  for (i in 1:nrow(new_data)) {
    a <- new_data$CategoryA[i]
    b <- new_data$CategoryB[i]
    table[a, b] <- table[a, b] + 1
  }
  return(table)
}

# -----

# 3. Functions for dynamic plotting
# -----

# Mosaic plot
plot_mosaic_dynamic <-
function(table, batch_num) {
  mosaic(
    table,
    shade = TRUE,
```

```

        legend = TRUE,

        main = paste("Dynamic Mosaic
Plot - Batch", batch_num))
    }

# Heatmap
plot_heatmap_dynamic <-
function(table, batch_num) {
    df <- melt(table)

    ggplot(df, aes(Var1, Var2, fill =
value)) +

        geom_tile(color = "grey70") +

        scale_fill_gradient(low =
"white", high = "steelblue") +

        labs(
            x = "Category A",
            y = "Category B",
            fill = "Count",

            title = paste("Dynamic Heatmap
- Batch", batch_num)) +

        theme_minimal()
    }

# -----
# 4. Simulate streaming data
# -----

set.seed(123)

# Number of batches to simulate
num_batches <- 20

for (i in 1:num_batches) {

    # Simulate a new batch of click
events

    new_events <- data.frame(

```

```

        CategoryA = sample(categories_A,
5, replace = TRUE),

        CategoryB = sample(categories_B,
5, replace = TRUE)

    )

    # Update contingency table

    cont_table <-
update_contingency(new_events,
cont_table)

    # Calculate chi-square & Cramer's
V

    chi_res <- chisq.test(cont_table)

    cramers_v <-
sqrt(chi_res$statistic /
(sum(cont_table) *
(min(dim(cont_table)) - 1)))

    cat(sprintf("Batch %d | Chi-sq:
%.2f | Cramer's V: %.2f\n",

                i, chi_res$statistic,
cramers_v))

    # -----
    # 5. Plot dynamically and save
    # -----

    # Mosaic plot

    mosaic_file <-
file.path(output_folder, paste0(i,
"mosaic.png"))

    png(mosaic_file, width = 800,
height = 600)

    plot_mosaic_dynamic(cont_table, i)

    dev.off()

    # Heatmap

```

```
    heat_plot <-  
    plot_heatmap_dynamic(cont_table, i)
```

```
    heat_file <-  
    file.path(output_folder, paste0(i,  
    "heat.png"))
```

```
    ggsave(  
        filename = heat_file,  
        plot = heat_plot,  
        width = 8,  
        height = 6,  
        dpi = 300  
    )
```

```
    print(heat_plot)
```

```
    Sys.sleep(1) # Pause to simulate  
    real-time arrival  
}
```

```
# -----  
# 6. Final contingency table  
# -----  
cont_table
```